

A | Final Thesis

Final Thesis – Base Python (Winter Semester 2025/2026)

This final assignment assesses your understanding of **basic Python concepts**, **logical thinking**, and your ability to **explain code in your own words**.

The tasks combine programming, data handling, and reflection. They are designed so that **understanding and explanation** are just as important as correct code.

Final Instructions for Comments and Code Quality

For **every task**, your code **must include comments** that:

- explain **what your code does**
- explain **why you chose this solution**
- are written in **your own words**
- reflect **your personal understanding**

AI-generated comments are NOT allowed.

Copy-paste explanations (from AI tools, websites, or other students) are **not acceptable**.

Your code may work perfectly — but **without clear, personal comments, you will lose points**.

Why comments matter

Programming is not only about producing correct output.
It is also about being able to:

- explain your logic
- describe your decisions
- show that you understand what happens step by step

In professional and academic contexts, **readable and well-commented code is essential**.

Comment Evaluation (applies to EVERY task)

For each task, you can earn:

- **+0.5 bonus points** for **clear, meaningful comments**
- Bonus points are **not included in the total score**,
but **can help you reach a higher grade** if you are close to a grade boundary.

Poor, generic, or AI-style comments may lead to **point deductions**.

Required Comment Structure

At the **top of each script or function**, include a short description like this:

```
"""
Description:
This script solves Task X of the final thesis.

It explains:
- what data is processed
- which logic is used
- what result is produced
"""
```

Good comments are rewarded!

For each task, you can receive an extra **0.5 points** for clear, well-structured comments.

These points are **not included in the main grade calculation**, but they can help you if you are just below a grade boundary.

For example: If you have 38 points (below passing level), but you wrote very good comments, you can receive enough extra points to pass (grade 4).

So it is always worth it to write clear and meaningful comments!

Task 1: Even and Odd Numbers (5 p.)

Write a program that loops through the numbers from 1 to 20 and prints whether each number is even or odd.

- 1 Create a loop from 1 to 20. (2 p.)
 - 2 Use an if-else statement to check even or odd. (2 p.)
 - 3 Print each number with the label “even” or “odd”. (1 p.)
-

Task 2: Sum of Even Numbers (6 p.)

Write a program that calculates and prints the sum of all even numbers from 1 to 1000.

- 1 Create a loop from 1 to 1000. (2 p.)
 - 2 Use a condition to select only even numbers. (2 p.)
 - 3 Calculate and print the final sum. (2 p.)
-

Task 3: D3TD5T (12 p.)

Write a program that loops through a list (the list will be given next week).

- Create a loop.
- If the element is a number and divisible by 3, print “D3T” inside the loop.
- If the element is a number and divisible by 5, print “D5T” inside the loop.
- If the element is a number and divisible by both 3 and 5, print “D3TD5T” inside the loop and save it in a new list called `d3td5t_list`.
- If the element is a string, check its first character. If it is “U”, save it in a new list called `find_strings_U`.

- Print all other numbers inside the loop.
- Finally, print the `d3td5t_list` and the `find_strings_U` list.

- 1 (2 p.)
- 2 (1 p.)
- 3 (1 p.)
- 4 (3 p.)
- 5 (3 p.)
- 6 (1 p.)
- 7 (1 p.)

Task 4: Replace Characters (6 p.)

Write a program that goes through a list of fruits (the list will be given next week) and replaces all occurrences of a specific character (e.g., “a”) in each word with another character (e.g., “o”).

- 1 Read/ initialize the list of fruits. (1 p.)
- 2 Loop through each fruit and replace a character. (4 p.)
- 3 Print each modified fruit name. (1 p.)

Task 5: Remove and Add Friends with Loop and Replace (6 p.)

Write a program that: - Loops through a list of friends (the list will be given next week). - Removes “Ben” if present. - Adds a new friend “Felix” at position 2. - Replaces “Anna” directly with “Bella”.

- 1 (2 p.)
- 2 (2 p.)
- 3 (1 p.)
- 4 (1 p.)

Task 6: Search for a last name in a DataFrame (15 p.)

Description:

In this task, you will practice iterating over a DataFrame and stopping a search as soon as a condition is met. This time, you will search only by last name.

Preparation:

- Create a string variable `my_lastname` containing your own last name.
- Create a boolean variable `found`, initialized as `False`.
- You will receive a CSV file with personal data next week. You do not need to create the DataFrame yourself!

Steps: - Read the given DataFrame - Find out which columns exist in the DataFrame. - Loop through the DataFrame row by row. - Check if the last name column in the current row matches `my_lastname`. - If a match is found, set `found = True` and **stop** the loop immediately. - After the loop, check `found`: - If the last name was not found, print:

No registration found for user {xxx}. You are not yet registered for the course "Base Python 2". - Replace {xxx} with your actual `my_lastname` variable.

1 (1 p.)
 2 (1 p.)
 3 (2 p.)
 4 (6 p.) 5 (5 p.) —

Task 7: Game — “Lucky String Duel: You vs Computer” (15 p.)

Description:

In this advanced mini-game, you will play a “lucky string draw” duel against the computer!

Imagine you both have access to a magical bag containing 100 surprise words stored in a DataFrame.

You will receive this DataFrame next week.

Hint: Think carefully about how you access the word from the DataFrame!

DataFrame Structure:

- Column Index: 1 to 100
- Column Word: string values

Steps: - Load the data - Define all needed variables to store the scores and counters for you and for the computer. - Define two separate loops: one for your turns and one for the computer’s turns. - In each loop, use a random “dice throw” (1–100) to select a word by index from the DataFrame, then calculate the results and update your counters: - Starts with “S” → +1 p. - Starts with “St” → +10 p. - Exactly “Strike” → +100 p. - Print all variables and counters clearly after both loops and the winner with a final message.

1 (1 p.)
 2 (1 p.)
 3 (4 p.)

4 (8 p.)

5 (1 p.)

Task 8: “Visualizing the Largest Cities in Europe Based on Coordinates” (15 p.)

Description:

You will work with the `worldcities.csv` file (SimpleMaps).

Instructions:

- 1 You will receive the download link to the World Cities dataset next week. Use the **free version**.
- 2 Unzip and **note the path** to `worldcities.csv`.
- 3 Load it into a pandas DataFrame and **print all column names**.
- 4 Filter cities in Europe using: - Latitude: 35° to 72° N. - Longitude: -25° to 45° E.
- 5 **Print the city names** from the filtered DataFrame.
- 6 Sort the DataFrame by **population** in descending order.
- 7 Create a **bar chart** (Matplotlib) for the top 10 largest cities: - X-axis: City names. - Y-axis: Population. - Add a descriptive title, e.g., “*Top 10 Largest Cities in Europe (by Population)*”.

1 (1 p.)

2 (1 p.)

3 (2 p.)

4 (4 p.)

5 (1 p.)

6 (2 p.)

7 (4 p.)

Bonus Task: Object-Oriented Programming with Vehicles (15 p.)

Description:

Create a base class `Vehicle` with common attributes such as `color`, `year`, `engine`, `name`, and `model`.

Subclasses:

- **Car (PKW):**
 - Add a new attribute `type` (e.g., “limousine”, “cabrio”, “coupe”).
 - Define a method `print_type()` that prints the car’s type.
 - **Truck (LKW):**
 - Add a new attribute `max_load_kg` that defines how many kilograms the truck can carry.
 - Define a method `print_max_load()` that prints this load capacity.
-

Data:

- You will receive **two CSV files next week**:
 - One file for cars (PKW), including an extra column `type`.
 - One file for trucks (LKW), including an extra column `max_load_kg`.
-

Steps:

- Define the base class **Vehicle** with attributes.
 - Define the subclass **Car** with the extra attribute and method.
 - Define the subclass **Truck** with the extra attribute and method.
 - Load both CSV files into separate DataFrames.
 - Create lists of **Car** and **Truck** objects from the respective DataFrames.
 - Print how many vehicles were found in total and how many of them are “VW”.
 - Create a bar chart (Matplotlib) showing the number of cars and trucks per model for vehicles older than 2000.
 - Add a legend with model names.
-

- 1 Define the base class **Vehicle** with common attributes. *(2 p.)*
 - 2 Define the subclass **Car** with extra attribute and method. *(3 p.)*
 - 3 Define the subclass **Truck** with extra attribute and method. *(3 p.)*
 - 4 Load both CSV files and create object lists. *(3 p.)*
 - 5 Print total vehicle count and “VW” count. *(2 p.)*
 - 6 Create a bar chart for vehicles older than 2000, with legend. *(2 p.)*
-

Final p. Overview

- Task 1: 5 p.
- Task 2: 6 p.
- Task 3: 12 p.
- Task 4: 6 p.
- Task 5: 6 p.
- Task 6: 15 p.
- Task 7: 15 p.
- Task 8: 15 p.
- Task 9: 15 p.
- Bonus Task: 15 p. (not included in final score)
- **Total (without Bonus): 80 p.**

Suggested grades

- From 42 p. = grade 4 (pass)
- From 52 p. = grade 3
- From 62 p. = grade 2
- From 72 p. = grade 1

Note about the Bonus Task

The points you collect from the bonus task are not included in the main grade calculation.

However, if your total points are just below a higher grade level, I will count your bonus task points to help you reach the better grade.

It is completely your choice whether you want to try the bonus task or not — but it can be a great opportunity to improve your final result!